

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA IN INFORMATICA

**Implementazione di modelli
TinyML su ESP32 per il rilevamento
di anomalie da dati vibrazionali con
infrastruttura backend centralizzata**

RELATORI:

Prof. Michele Pasqua

LAUREANDO:

Secchi Pietro Giampaolo

ANNO ACCADEMICO 2025 - 2026

Indice

Introduzione	1
1 Analisi del Contesto e Requisiti	3
1.1 Cinematica delle Vibrazioni Meccaniche	3
1.1.1 Origine delle Vibrazioni	3
1.1.2 Limiti dell'analisi nel dominio del tempo	4
1.2 Anomaly Detection su un singolo regime	5
1.3 Anomaly Detection a Regimi Multipli	5
1.4 Vincoli Computazionali	5
1.5 Specifica dei Requisiti di Sistema	6
1.5.1 Requisiti Funzionali	6
1.5.2 Requisiti Non Funzionali	7
2 Architettura Generale del Sistema	9
2.1 Panoramica dell'Architettura di Sistema	9
2.2 Scelta del Protocollo di Rete	10
2.2.1 HTTP	10
2.2.2 MQTT	10
2.3 Scelta del Protocollo di Presentazione Dati	11
2.3.1 Protocolli basati sul testo	11
2.3.2 Protocolli basati su Rappresentazione Binaria	12
2.4 Architettura backend	14
2.5 Architettura Edge/Firmware	15
3 Sviluppo del Livello Edge	17
3.1 Hardware	17

3.1.1	Consumo Energetico	19
3.1.2	Sintesi dei Consumi di Sistema	22
3.2	Software	22
3.2.1	Struttura principale	22
3.2.2	4-way Buffer	24
3.3	Inferencer	26
3.3.1	Architettura dell' <i>ensemble</i>	28
3.3.2	Gestione dei sottomodelli	28
3.4	Ottimizzazioni	29
3.4.1	Ottimizzazione della conversione in float	29
4	Modelli utilizzati	33
4.0.1	Modello Memoria	33
5	Sviluppo del Backend e Integrazione	37
5.1	Pipeline dei Dati	37
5.1.1	Topic MQTT	37
5.1.2	Persistenza dei Dati	38
5.1.3	NAS Engine	39
6	Testing, Deployment e Risultati	43
6.1	Benchmark	43
6.1.1	Velocità	43
6.1.2	Accuratezza e Valutazione Benchmark	44
6.1.3	Occupazione di Memoria RAM e Flash	46
7	Conclusioni e Sviluppi Futuri	49
7.1	Conclusioni	49
7.1.1	Quando la Memoria Peggiora	49
7.1.2	Quando un Ensemble Eterogeneo Peggiora	50
7.2	Sviluppi Futuri	52
7.2.1	Miglioramento dei modelli	52
7.2.2	Miglioramento Architettura del Sistema	53
	Bibliografia	55

Introduzione

Fin dalle rivoluzioni industriali, con i primi macchinari, le fabbriche hanno dovuto gestire i guasti di questi ultimi. Un guasto comporta ritardi alla produzione e relative perdite economiche. Nella storia si è passati dalla manutenzione reattiva, ovvero riparare i macchinari dopo un guasto effettivo, alla manutenzione preventiva programmata, ovvero sostituire le parti più soggette ad usura ad intervalli regolari o a ore-macchina, alla manutenzione predittiva, ovvero sostituire componenti basandosi su misurazioni che possono essere di vibrazione, rumore o temperatura. Questa tesi si concentra sulla manutenzione predittiva. Le prime modalità di manutenzione predittiva prevedevano l'uso di figure specializzate, chiamate in inglese "machine whisperers" (coloro che sussurrano alle macchine). Questi dipendenti analizzavano i macchinari durante il funzionamento per notare anomalie. Erano molto specializzati, e di conseguenza costosi, anche se per le aziende rappresentavano un costo marginale rispetto ai fermi di produzione e ai costi di riparazione. A partire dall'Industria 3.0, con i primi macchinari computerizzati si è iniziato ad aggiungere sensori vibrazionali per estrarre informazioni utili a prendere decisioni riguardo alla manutenzione, ma questi sistemi erano ancora dipendenti da persone che analizzassero i dati o da soglie fisse e algoritmi euristici. L'utilizzo di intelligenze artificiali è stato introdotto con l'Industria 4.0 per l'analisi dei dati. Questi dati vengono trasportati nel cloud, su server remoti, per analizzare i dati storici, tuttavia trasmettere dati grezzi ad alta frequenza verso il cloud satura la banda di rete, introduce latenza e richiede un notevole dispendio energetico e di risorse. Per questo motivo si è iniziato nell'Industria 5.0 a portare l'intelligenza artificiale direttamente nei sensori (*Edge Computing/TinyML*). Secondo la Commissione Europea (rapporto del 2021: "Industry 5.0: Towards a sustainable, human-centric and resilient European industry" [Breque et al., 2021]), l'Industria 5.0 poggia su tre pilastri cardine: Sostenibilità,

Resilienza e Uomo al centro. Un approccio TinyML aumenta la sostenibilità spostando il calcolo su microcontrollori che costano pochi euro, consumano elettricità nell'ordine dei milliwatt e riducono la trasmissione dei dati tra sensore e cloud, liberando banda. La resilienza aumenta grazie alla capacità del microcontrollore di salvare i dati su una scheda microSD; questo permette all'intero sistema di continuare a monitorare le vibrazioni anche se la connessione cade, per poi caricare in un secondo momento i dati e i metadati verso il cloud. L'uomo rimane comunque centrale al processo decisionale: il servizio non lo sostituisce con una scatola nera, ma funge da strumento di diagnosi precoce, mettendo al primo posto la sicurezza sul lavoro e rilevando in anticipo possibili guasti pericolosi. Inoltre, l'operatore può gestire i dati contrassegnandoli dalla UI se riconosce una condizione specifica, guidando il servizio a creare modelli migliori. Tuttavia usare un unico modello con un'architettura statica per calcolare le anomalie può risultare controproducente: potrebbe fare fatica a notare anomalie se i dati sono molto eterogenei e potrebbe non essere la migliore architettura per il tipo di dati specifico. Per questo si usano algoritmi NAS (*Neural Architecture Search*), che confrontano varie architetture e dimensioni dei modelli per decidere la più performante. In questa tesi si esplora un'applicazione avanzata di NAS, che unisce tipi diversi di architetture nello stesso gruppo (*ensemble*) con un modello che orchestra diversi sottomodelli; questo permette di scegliere per un macchinario diversi modelli specializzati in stati diversi del macchinario stesso, rendendo il sistema più efficiente ed efficace. Il lavoro inoltre esplora lo sviluppo di un dispositivo capace di raccogliere dati vibrazionali, caricarli nel cloud e ricevere modelli con cui fare l'inferenza sui dati per trovare anomalie; viene inoltre esplorata l'intera infrastruttura cloud a supporto del dispositivo, inclusi la scelta e l'allenamento dei modelli.

Capitolo 1

Analisi del Contesto e Requisiti

1.1 Cinematica delle Vibrazioni Meccaniche

1.1.1 Origine delle Vibrazioni

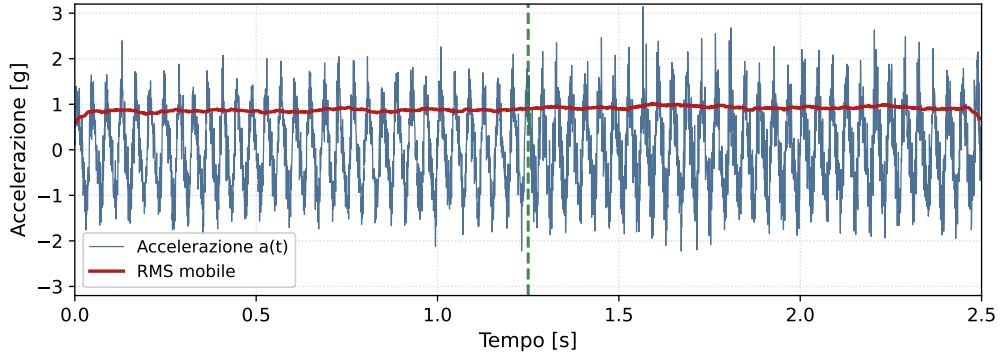
In un macchinario le vibrazioni vengono principalmente da organi rotanti, quali mandrini, motori e pompe. I guasti di un organo rotante ne cambiano la sequenza caratteristica nei seguenti modi:

- **Sbilanciamento dell'albero rotante** causato spesso da uno squilibrio di massa, genera un picco alla frequenza di rotazione dell'organo.
- **Disallineamento angolare o parallelo** produce armoniche al doppio della frequenza di rotazione.
- **Difetti nei cuscinetti** producono frequenze di impatto caratteristiche alla loro velocità di rotazione calcolabili.

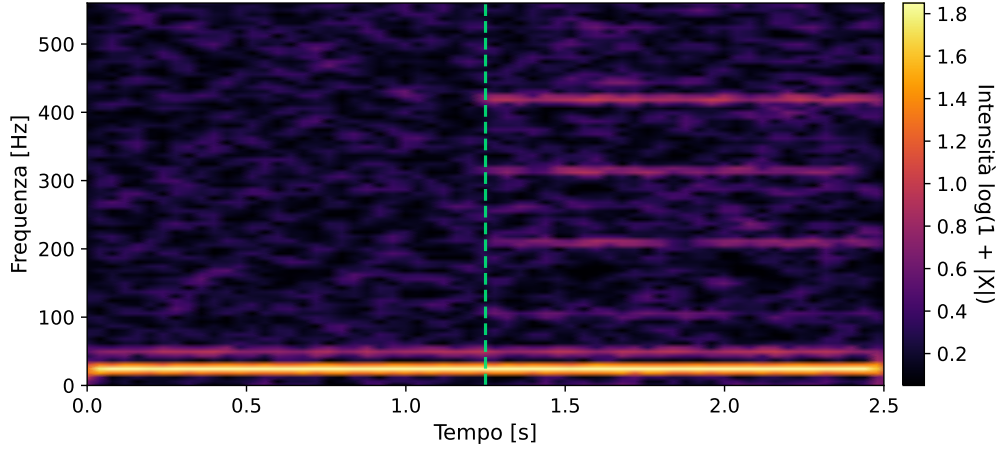
Per raccogliere questi dati vibrazionali si ha bisogno di un sensore; i primi sensori vibrazionali erano trasduttori piezoelettrici analogici, che sono costosi, ingombranti e necessitano di un circuito analogico per la lettura dei dati. Con l'Industria 4.0 sono nati i primi sensori MEMS (micro-electromechanical systems), circuiti integrati digitali che sono economici, poco ingombranti, non hanno bisogno di schermature dei cavi per la natura digitale del segnale, e molti permettono anche la raccolta di dati in più di un asse; il sensore usato nel progetto è a 6 assi (3 per l'accelerazione e 3 per il giroscopio) [STMicroelectronics, 2024a].

1.1.2 Limiti dell'analisi nel dominio del tempo

Ci sono vari metodi statistici nel dominio del tempo per trovare anomalie nelle vibrazioni come RMS, valore di picco, fattore di cresta o curtosi, tuttavia questi metodi non sono efficaci a localizzare guasti a bassa energia in fase nascente. Per questo passiamo al dominio delle frequenze usando trasformazioni di Fourier a breve termine (STFT) [Lyons, 2010], che consentono di osservare il cambio delle frequenze caratteristiche al variare del tempo.



(a) Segnale con anomalia nel dominio del tempo



(b) Segnale con anomalia nel dominio delle frequenze STFT

Figura 1.1: Confronto tra l'analisi temporale 1.1a e la trasformata STFT 1.1b in presenza di anomalia.

1.2 Anomaly Detection su un singolo regime

L'anomaly detection ha un problema fondamentale nello sviluppo, ovvero l'asimmetria dei dati: i macchinari funzionano senza anomalie per la maggior parte della loro vita, questo implica che durante lo sviluppo di meccanismi di anomaly detection non si hanno esempi di dati anomali, quindi bisogna basarsi completamente su dati non anomali senza dati etichettati di anomalie. Per risolvere questo problema si usano modelli AI chiamati autoencoder, usando apprendimento non supervisionato. Durante la fase di apprendimento il modello viene addestrato esclusivamente con dati non anomali con lo scopo di ricostruirli in maniera più fedele possibile; tuttavia se ne limita nell'architettura la potenza espressiva negli strati centrali. In questo modo impara a ricostruire segnali specifici, quelli non anomali, e se dopo l'apprendimento si prova a fornirgli dati anomali, esso proverà a ricostruirli cercando di farli ricadere nella struttura dei dati non anomali. Per quantificare l'anomalia si controlla la differenza tra l'input e l'output: se i dati in uscita sono molto diversi dai dati in ingresso significa che quei dati il modello non li ha visti durante l'apprendimento.

1.3 Anomaly Detection a Regimi Multipli

Molti macchinari hanno più di un regime, come una pompa che varia velocità in base alla pressione sulla linea: in questi casi, se provassimo ad usare modelli mono-regime per poter riconoscere regimi diversi, avremmo bisogno di modelli molto più grandi, pesanti e meno efficaci. In questa tesi si usa un'architettura multimodale, ovvero al posto di usare un singolo modello molto pesante o impossibile da caricare su un microcontrollore, se ne usano molti piccoli specializzati sul loro regime, con un modello router che decide quale sottomodello deve riconoscere l'anomalia.

1.4 Vincoli Computazionali

Il calcolo delle anomalie avviene su microcontrollori con ridotta potenza di calcolo (dual-core a 240 MHz), che eseguono un sistema operativo real-time (FreeRTOS), molto più leggero rispetto a un sistema operativo desktop. L'assenza di MMU implica che un crash di memoria blocchi l'intero sistema obbligandolo al riavvio,

e la memoria a disposizione è limitata (all'incirca 8 MB con PSRAM); questo rende impossibile usare le architetture neurali comunemente impiegate su server, imponendo di comprimere i modelli per poter funzionare nei vincoli del dispositivo. L'ottimizzazione è il pilastro principale del sistema. Per l'inferenza dei modelli si usa TensorFlow Lite for Microcontrollers (TFLite Micro) [David et al., 2021, Espressif Systems, 2024d], che consente di includere nel compilato unicamente il sottoinsieme strettamente necessario degli operatori Keras/TensorFlow. Inoltre, il microcontrollore scelto (ESP32-S3) dispone di estensioni vettoriali SIMD e di pipeline DSP (Digital Signal Processing) ottimizzate tramite librerie fornite dal produttore. Come detto precedentemente, forniamo segnali nel dominio delle frequenze ai modelli, perché più espressivi.

1.5 Specifica dei Requisiti di Sistema

1.5.1 Requisiti Funzionali

- **Acquisizione Continua:** campionamento a 6 assi (accelerometro e giroscopio) con garanzia di zero packet drop tramite FIFO SPI e interrupt hardware.
- **Estrazione Spettrale STFT:** calcolo locale di spettrogrammi (finestra Hann, overlap 75%, compressione logaritmica) direttamente sul microcontrollore.
- **Inferenza Multi-Modello (Ensemble):** classificazione del regime operativo tramite modello Router e selezione dinamica dell'Autoencoder dedicato, con eventuale supporto a modelli con memoria temporale.
- **Rilevamento e Notifica delle Anomalie:** calcolo del punteggio di ricostruzione (MSE lineare o logaritmico), confronto con soglia configurabile e notifica dell'anomalia.
- **Gestione Telemetria e Connettività MQTT:** trasmissione asincrona di messaggi telemetrici (campioni grezzi, spettrogrammi, esiti di inferenza) verso il broker MQTT centrale.

- **Archiviazione di Sicurezza Locale:** salvataggio continuo o su evento anomalo dei segmenti vibrazionali su memoria microSD per tollerare cadute della connettività di rete.
- **Aggiornamento OTA dei Modelli:** ricezione e installazione di nuovi modelli neurali e configurazioni distribuiti dal backend senza richiedere riprogrammazione fisica del chip.
- **Pipeline MLOps Centralizzata:** ingestione continua dei dati, archiviazione su database, monitoraggio visivo e ricerca automatizzata delle architetture neurali.

1.5.2 Requisiti Non Funzionali

- **Determinismo Temporale:** la quantità di dati calcolati (acquisizione dati + STFT + Router + Autoencoder) deve essere in media uguale alla quantità di dati generati dal sensore MEMS.
- **Vincoli di Memoria e Allocazione Statica:** il firmware deve rispettare i vincoli di SRAM interna (< 512 KB) per task e dati critici e PSRAM esterna (8 MB) per tensor arena e buffer audio/vibrazionali.
- **Efficienza Energetica:** gestione di modalità a basso consumo (modem sleep Wi-Fi e duty-cycle).
- **Efficienza di Protocollo e Banda:** impiego della serializzazione binaria CBOR, con riduzione dell'overhead di payload rispetto al formato testuale JSON.
- **Robustezza e Tolleranza ai Guasti:** isolamento dei task critici su core dedicati di FreeRTOS (Core 1 realtime, Core 0 I/O bound), riconnessione automatica al broker MQTT e salvaguardia della coerenza del filesystem FATFS su microSD.

Capitolo 2

Architettura Generale del Sistema

2.1 Panoramica dell'Architettura di Sistema

Il sistema è stato sviluppato con un'architettura distribuita di tipo *edge-to-cloud*, progettata per disaccoppiare le fasi di acquisizione dati vibrazionali, l'inferenza di modelli *on edge* per il rilevamento delle anomalie e l'addestramento di questi modelli. Il sistema è formato da quattro livelli principali:

- **edge**: dispositivo formato da un microcontrollore e un sensore MEMS a 6 assi, acquisisce i dati vibrazionali, li emette sporadicamente al cloud ed esegue l'inferenza di modelli per valutare se i dati sono anomali.
- **rete**: i dati vengono trasmessi *wireless* usando il protocollo MQTT.
- **backend**: allena i modelli basandosi sui dati ricevuti dai sensori.
- **storage**: abilita la persistenza dei dati storici.

2.2 Scelta del Protocollo di Rete

Sono stati valutati due protocolli per la trasmissione dei dati, questi sono **HTTP** e **MQTT**.

2.2.1 HTTP

- **Vantaggi:**

- Ideale per l'integrazione con servizi tradizionali client-server.
- Permette trasferimento di file di grandi dimensioni attraverso meccanismi di *chunking*.
- Ha una latenza più bassa rispetto a MQTT nel trasferimento di grandi quantità di dati.

- **Svantaggi:**

- **Overhead elevato:** gli *header* di ciascun pacchetto introduce un consumo maggiore di risorse rispetto a MQTT [Adriansyah et al., 2026].
- Poco adatto a sistemi embedded o dispositivi con risorse limitate di memoria e calcolo.

2.2.2 MQTT

- **Vantaggi:**

- **Efficienza:** l'architettura *publish/subscribe* e l'overhead ridotto (header fino a 2 byte) lo rendono ottimale per invii di dati sporadici.
- **Affidabilità:** la presenza di tre livelli di Quality of Service (QoS) garantisce la corretta consegna dei messaggi anche con connessioni instabili.
- Basso impatto computazionale e consumo energetico ridotto.

- **Svantaggi:**

- **Gestione limitata di file pesanti:** non è concepito per il trasferimento efficiente di payload di grandi dimensioni.

- **Necessario Broker:** necessita di un broker che gestisca la comunicazione tra gli endpoint.
- **Latenza maggiore:** l'intermediazione del broker e i meccanismi di QoS, soprattutto di livello 1 o 2 introducono ritardi rispetto a comunicazioni dirette.

Per la leggerezza del protocollo e per il sistema a *publish/subscribe* è stato scelto MQTT per tutte le comunicazioni tra l'edge device e il servizio. La parte più critica è la trasmissione dei modelli, essendo di dimensione non trascurabile, per questo tipo di comunicazione si sarebbe potuta usare un'architettura ibrida dove via MQTT l'edge device viene notificato del caricamento dei nuovi modelli che verrebbero poi scaricati via una richiesta GET di HTTP usando le funzioni di *chunking* del payload.

2.3 Scelta del Protocollo di Presentazione Dati

È necessaria una Lingua Franca per la rappresentazione dei dati condivisa che permetta ai vari dispositivi nella rete di codificare e decodificare i dati in un formato condiviso, come descrive il livello 6 del modello *ISO/OSI* [International Organization for Standardization, 1994]. Le scelte si dividono in protocolli basati su testo (XML [Bray et al., 2008], JSON [Bray, 2017]) e protocolli a rappresentazione binaria (CBOR [Bormann and Hoffman, 2020], BSON [MongoDB, Inc., 2009], Protocol Buffers [Varda, 2008], MessagePack [Furuhashi, 2013], Avro [Apache Software Foundation, 2011]).

2.3.1 Protocolli basati sul testo

XML nasce come linguaggio di marcatura per documenti complessi basato su tag ed elementi ricchi di metadati. Il JSON è un formato minimale per lo scambio di dati basato su coppie chiave-valore e array. Il JSON risulta notevolmente più leggero (30-40% di dimensione in meno [McNally, 2025]), umanamente più leggibile e rapido da elaborare via codice rispetto alla prolissità e alla complessità strutturale dell'XML.

2.3.2 Protocolli basati su Rappresentazione Binaria

Di natura sono più leggeri rispetto ai protocolli basati sul testo, evitando la conversione del testo in dati usabili dall'applicativo, quindi sono preferibili in sistemi limitati che giovano ad avere un risparmio di tempo di calcolo e di memoria.

Tabella 2.1: Confronto formati di serializzazione binaria

Caratteristica	CBOR	BSON	Protobuf	MessagePack	Avro
Origine / Focus	IETF / IoT e risorse minime	MongoDB / Storage e traversal DB	Google / RPC: velocità e peso	Furuhashi / Alternativa JSON	Apache / Schema evolution
Schema	Opzionale	Opzionale	Obbligatorio (.proto)	Opzionale	Obbligatorio (JSON/IDL)
Posizione Schema	Assente o est. (CDDL)	Assente	File .proto separato	Assente	Nei file / Registry
Auto-descrittivo	Sì	Sì	No	Sì	No (serve schema)
Codifica base	JSON + Tag	JSON esteso	Tag / ID numerici campo	JSON + tipi ext	Ordine campi schema
Estensibilità	Tag IANA	Tipi custom DB	Estensioni/ opzioni proto	Tipi ext	Regole evoluzione
Evoluz. Schema	Implicita (salto tag)	Implicita	Esplicita (stabilità ID)	Implicita (salto ext)	Esplicita (alias/default)
Dimensione	Compatta	Variabile (anche grande)	Molto compatta	Compatta	Molto compatta
Velocità	Alta	Alta (traversal DB)	Massima (RPC)	Alta	Alta
Standard	IETF RFC 8949	De facto (MongoDB)	De facto (Google)	Community spec	Apache Project
Usi Principali	IoT, CoAP, sicurezza	MongoDB	RPC, Microservizi	Caching, RPC, stream	Big Data (Kafka, Hadoop)

Fonte: [McNally, 2025].

Data la tabella la migliore scelta è *CBOR* (Concise Binary Object Representation), per i seguenti motivi: è un protocollo *royalty-free* e *open source*, è stato sviluppato dalla base per essere usato in contesti IoT per usare poca memoria e supporta nativamente la crittazione dei dati, attraverso lo standard COSE (CBOR Object Signing and Encryption, RFC 9052 [Schaad, 2022]).

Viene usata come libreria sul microcontrollore *espressif/cbor* [Espressif Systems, 2024f], derivata da *tinycbor* [Intel Corporation and Macieira, 2015], ed è impiegato anche *zcbor* [Øyvind Rønningstad and Nordic Semiconductor, 2021], un programma che permette di tradurre lo schema CDDL [Birkholz et al., 2019] direttamente in codice

C ottimizzato per la codifica e/o decodifica dei pacchetti in struct.

2.4 Architettura backend

Il backend è formato da servizi ospitati su container Docker [Merkel, 2014], il che permette di rendere il servizio modulare e facilmente riproducibile. I servizi docker sono:

- **emqx**: [EMQ Technologies Inc., 2024] Un broker MQTT, versione Community, gestisce la connessione MQTT.
- **postgres**: [PostgreSQL Global Development Group, 2024] un database relazionale con estensione TimescaleDB [Timescale, Inc., 2024], che migliora le prestazioni della base di dati nella gestione, scrittura e analisi di dati temporali.
- **api_service**: Server FastAPI [Ramírez, 2018] per la gestione dei dati del database.
- **mqtt_service**: Si interfaccia al servizio MQTT, ricevendo i dati dal sensore e pubblicandogli i modelli e configurazioni, gestendone il lifecycle.
- **nas_service**: Neural Architecture Search, Servizio che usa Optuna [Akiba et al., 2019] per la ricerca architetturale e allenamento modelli tflite [David et al., 2021] specifici al sensore.
- **grafana**: [Grafana Labs, 2024] Dashboard per visualizzazione dei dati e risultati di inferenza.
- **clearml_***: [Allegro AI, 2024] Piattaforma per tracciare parametri, loss e metriche dei modelli, divisa in 6 container:
 - API
 - Web
 - Files
 - Elastic
 - Redis
 - Mongo

2.5 Architettura Edge/Firmware

Per garantire reattività nell’acquisizione, l’inferenza e l’invio dei dati, il firmware è strutturato su un modello concorrente che sfrutta l’architettura dual-core del microcontrollore, ovvero un ESP32-S3, scelto per le alte prestazioni.

Come sensore MEMS viene usato il modulo *STEVAL-MKI245KA* [STMicroelectronics, 2024b], di STMicroelectronics, un kit di valutazione del circuito integrato *ISM330BX* [STMicroelectronics, 2024a], concesso in uso per le attività di ricerca da Iotinga SRL.

Il sistema operativo usato è *FreeRTOS* [Barry and Amazon Web Services, 2022], un sistema operativo runtime minimale che espone le funzionalità base di un sistema operativo, come la creazione di task, la gestione della rete e del *file system*.

Come *framework* di sviluppo viene usato ESP-IDF [Espressif Systems, 2024b], scelto per il controllo più a basso livello, rispetto ad Arduino Core, delle periferiche hardware e la gestione nativa dei task FreeRTOS.

Capitolo 3

Sviluppo del Livello Edge

3.1 Hardware

Vengono usati:

- **ESP32-S3** [Espressif Systems, 2024e] con modulo PSRAM da 8MB e memoria flash da 16MB.
- **STEVAL-MKI245KA** [STMicroelectronics, 2024b] in modalità di comunicazione SPI.
- **Adafruit 4682** [Adafruit Industries, 2020] lettore sd in modalità di comunicazione SPI.

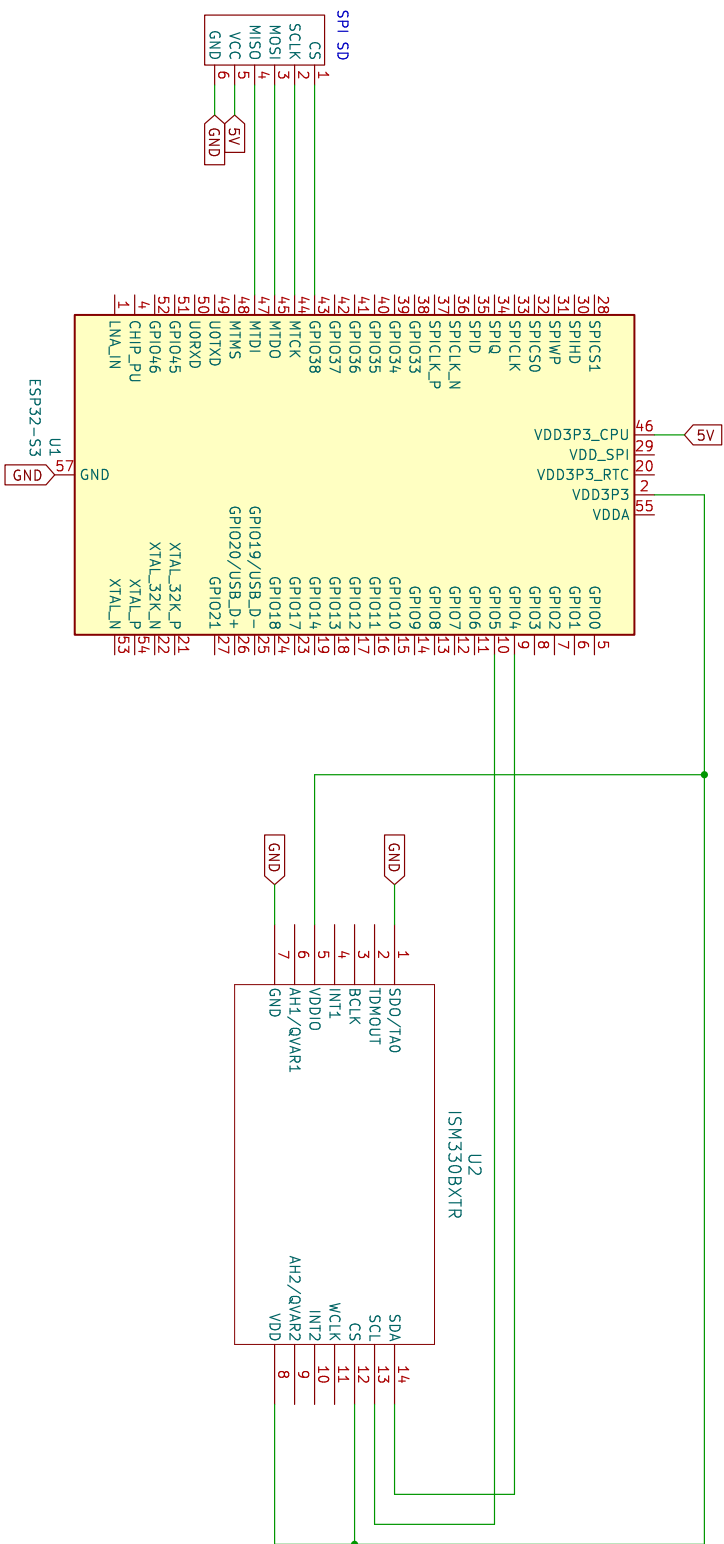


Figura 3.1: Schema logico

3.1.1 Consumo Energetico

ESP32-S3 (Microcontrollore)

Tabella 3.1: ESP32-S3 Power Consumption – CPU Active, 25 °C

CPU Frequency	Peripherals	Data Access	Consumption
80 MHz	×	32 bit	33.1 mA
80 MHz	✓	32 bit	47.3 mA
80 MHz	×	128 bit	41.8 mA
80 MHz	✓	128 bit	56.3 mA
240 MHz	×	32 bit	66.2 mA
240 MHz	✓	32 bit	81.3 mA
240 MHz	×	128 bit	91.7 mA
240 MHz	✓	128 bit	107.9 mA

Tabella 3.2: ESP32-S3 Low-Power Modes, 25 °C

Mode	Condition	Consumption
Light-sleep	CPU e digital peripherals in clock-gating, Wi-Fi disattivato	0.240 mA
Deep-sleep	RTC memory e RTC peripherals alimentati	0.008 mA

Tabella 3.3: ESP32-S3 Wi-Fi Power Consumption, 25 °C

Mode	Configuration	Consumption
Transmitter	802.11b, 1 Mbps, $P_{OUT} = +21$ dBm	340 mA
Transmitter	802.11g, 54 Mbps, $P_{OUT} = +19$ dBm	291 mA
Transmitter	802.11n, HT20, MCS7, $P_{OUT} = +18.5$ dBm	283 mA
Transmitter	802.11n, HT40, MCS7, $P_{OUT} = +18$ dBm	286 mA
Receiver	802.11b/g/n, HT20	88 mA
Receiver	802.11n, HT40	91 mA

Il profilo energetico usato é *Modem-sleep mode*, rilasciato da Espressif [Espressif Systems, 2024e], dove il modem viene attivato solo quando necessario, quindi il consumo effettivo é:

- in **Modem TX**: CPU Active/All RTC/Wi-Fi, durante la comunicazione dei dati (286 mA).
- in **Modem RX**: CPU Active/All RTC/Wi-Fi, durante la ricezione di modelli, comandi e configurazioni dei dati (91 mA).

- in **Modem sleep**: CPU Active/All RTC/No Wi-Fi, durante la fase di sola inferenza, scrittura su sd o lettura del sensore (81.3 mA).

Il sistema acquisisce campioni da 12 byte (16bit per valore, 3 per accelerometro e 3 per giroscopio) ad una frequenza di 3.84 kHz (46.08 kB/s). I dati, convertiti in *float32* (24 byte per campione), vengono trasmessi in blocchi da 4096 campioni. La dimensione di ciascun blocco è quindi

$$N_{\text{TX}} = 4096 \cdot 24 = 98304 \text{ byte.} \quad (3.1)$$

Il periodo tra due trasmissioni in modalità *continua*, ovvero vengono caricati tutti i *datapoint*

$$T_{\text{block}} = \frac{4096}{3840} = 1.067 \text{ s.} \quad (3.2)$$

Assumendo la velocità teorica massima di 150 Mbit/s per 802.11n HT40 MCS7, il tempo necessario per un blocco è

$$T_{\text{TX}} = \frac{98304 \cdot 8}{150 \cdot 10^6} = 5.24 \text{ ms.} \quad (3.3)$$

Assumendo una corrente di 286 mA la potenza per singola trasmissione è

$$E_{\text{TX}} = \frac{3.3 \cdot 0.286 \cdot 5.24 \times 10^{-3}}{3.6} \approx \boxed{1.37 \times 10^{-3} \text{ mWh}}. \quad (3.4)$$

Il duty cycle della trasmissione è

$$D_{\text{TX}} = \frac{5.24 \text{ ms}}{1.067 \text{ s}} \approx 0.492\%. \quad (3.5)$$

La corrente media associata al Wi-Fi è quindi

$$I_{\text{WiFi,avg}} = 286 \text{ mA} \cdot 0.00492 \approx 1.41 \text{ mA.} \quad (3.6)$$

Considerando inoltre 81.3 mA per CPU e periferiche attive, la corrente media complessiva risulta

$$I_{\text{avg}} = 81.3 + 1.41 \approx 82.7 \text{ mA.} \quad (3.7)$$

I valori ottenuti rappresentano una stima teorica basata sulla velocità massima di 150 Mbit/s e non tengono conto degli overhead del protocollo Wi-Fi e dei livelli superiori.

ISM330BX

Modalità Operativa	Dettagli Funzionalità	Canali e	Corrente Tipica (I_{DD})
Combo High-Performance	Accelerometro + Giroscopio attivi ad alto ODR		$\approx 0.60 \text{ mA}$ ($600 \mu\text{A}$)
Power-Down	Dispositivo spento, interfacce seriali in attesa		$3 \mu\text{A} - 5 \mu\text{A}$

Tabella 3.4: Consumi elettrici ISM330BX [STMicroelectronics, 2024a].

Il sensore viene usato in modalità *High-Performance* a 3.84kHz, con coda FIFO, ovvero il sensore salva in una regione di memoria le letture, il che permette di leggere a blocchi i dati dal sensore. Il firmware interroga il sensore chiedendo 512 byte alla volta.

Adafruit 4682

Il breakout Adafruit 4682 è una scheda passiva, quindi il consumo è dato unicamente dalla scelta della scheda microSD. Consumo scheda SD:

Current	V_{DD}	Max.
Peak	3.6 V	300 mA
Average	3.6 V	250 mA

Tabella 3.5: consumi di una scheda microSD, microSDXC, Class 10, U1, 64GB, modello SDCIT/64GB, marca Kingston [Kingston Technology, 2024].

Analogamente al caso precedente, si riutilizzano gli stessi risultati per il consumo del controllore. Considerando un data rate di 46.08 kB/s, e una velocità minima di scrittura di 10 MB/s per una SD Classe 10, il duty cycle della scrittura o lettura è pari a circa 0.46%.

Assumendo un consumo di 250 mA durante la scrittura o lettura il consumo medio della SD è quindi circa 1.15 mA. Sommando il consumo della CPU e delle periferiche, pari a 81.3 mA, si ottiene

$$I_{\text{avg,SD}} \approx 82.45 \text{ mA}, \quad P_{\text{avg,SD}} \approx \boxed{272.1 \text{ mW}}. \quad (3.8)$$

3.1.2 Sintesi dei Consumi di Sistema

Sommando i consumi si ha un consumo medio teorico di $\approx 83.75 \text{ mA}$, o $\approx 276.4 \text{ mW}$, e un consumo di picco di $\approx 618.9 \text{ mA}$ o $\approx 2042 \text{ mW}$.

Il che significa che una batteria 18650 da circa 9Wh, riuscirebbe ad alimentare il dispositivo per 32 ore.

Tecniche di Ottimizzazione energetica

La stima di 32 ore si riferisce all'autonomia con il massimo consumo, ovvero con continua lettura, salvataggio, invio dei dati e l'inferenza basata su suddetti dati. Una volta che la *backend* riceve abbastanza dati per allenare modelli validi si può mettere il device in varie modalità risparmio energetico configurabile.

Modalità	CPU	Duty Cycle	Corrente	Potenza	Autonomia
Active	240 MHz	100%	83.75 mA	276.4 mW	32.6 h
Active	80 MHz	100%	49.75 mA	164.2 mW	54.8 h
Light-sleep	240 MHz	3.33%	3.70 mA	12.22 mW	30.7 giorni
Light-sleep	80 MHz	3.33%	2.57 mA	8.48 mW	44.2 giorni
Deep-sleep	240 MHz	3.33%	3.48 mA	11.47 mW	32.7 giorni
Deep-sleep	80 MHz	3.33%	2.34 mA	7.74 mW	48.5 giorni

Tabella 3.6: Consumi e autonomia stimati nelle modalità, batteria da 9 Wh. Il duty cycle è del 3.33% (2 minuti di attività ogni ora, ovvero circa 112 segmenti da 4096 datapoint). Il duty cycle del Wi-Fi viene scalato a 0.41%, il lettore SD è spento durante le fasi di sleep.

3.2 Software

3.2.1 Struttura principale

Il codice del firmware é scritto in C/C++ usando il framework ESP-IDF. Il sistema giova dell'architettura dual core del microcontrollore ESP32-S3 [Espressif Systems, 2024e]. Ci sono 6 task/thread principali.

Nome Task	Priorità	Core Assegnato
harvester	5	Core 1
inferencer	4	Core 1
uploader	3	Core 0
dump_reader	3	Core 0
sd_writer	2	Core 0
mqtt_task	5 (Default)	Core 0 (Default)

Tabella 3.7: Priorità delle Task FreeRTOS e selezione Core

Tutte le task nel core 0 hanno grandi ritardi di I/O perché richiedono chiamate di rete oppure la lettura e scrittura di file su scheda SD. Mentre l'inferencer nel Core 1 è cpu-bound per la maggior parte del tempo e IO-bound se deve caricare i modelli dalla rete oppure dalla scheda SD. L'harvester viene messo nello stesso Core dell'inferencer perché vogliamo dare precedenza alla lettura dei dati in maniera prioritaria.

harvester (Core 1, Priorità 5) Esegue l'interrogazione della coda FIFO del sensore ISM330BX via SPI per estrarre i campioni di accelerometro e giroscopio. La priorità è massima per permettere alla coda FIFO di non riempirsi mai e non andare in *overflow* [STMicroelectronics, 2024a], il che significherebbe perdere dei *datapoints* contigui.

inferencer (Core 1, Priorità 4) Calcola la magnitudo e carica dello spettrogramma STFT 2D. Esegue i modelli dell'*ensemble* (insieme di modelli) e calcola il valore delle anomalie.

uploader (Core 0, Priorità 3) Gestisce la serializzazione in formato CBOR e la trasmissione dei dati. Divide l'array di campioni float in pacchetti più piccoli per rientrare nei limiti dei buffer di rete dell'MQTT, e li invia al broker MQTT.

dump_reader (Core 0, Priorità 3) Viene istanziato temporaneamente in caso di trigger di replay, ovvero se la backend richiede un upload di tutti i dati storici salvati. Legge i file binari dalla scheda SD e li reinserisce nella pipeline di elaborazione, nello stesso modo in cui farebbe *harvester*, che viene messo in pausa nel caso venga richiesto il replay. È un processo IO-bound, quindi messo nel Core 0.

`sd_writer` (**Core 0, Priorità 2**) Prende i dati (sia i datapoint che i risultati dell'inferenza) e li scrive sulla scheda SD in formato binario.

`mqtt_task` (**Core 0, Priorità 5 – Default di Sistema**) Gestito in background dalla libreria client di MQTT di Espressif (*espressif/mqtt* [Espressif Systems, 2024c]).

3.2.2 4-way Buffer

La struttura dati principale che unisce le varie task é un *4-way buffer*, una variante di un *Ping Pong buffer* [Lyons, 2010] con 4 buffer al posto di 2. Tutte le task leggono e scrivono dagli slot del quad buffer, usando dei semafori per la sincronizzazione, e per evitare controlli attivi.

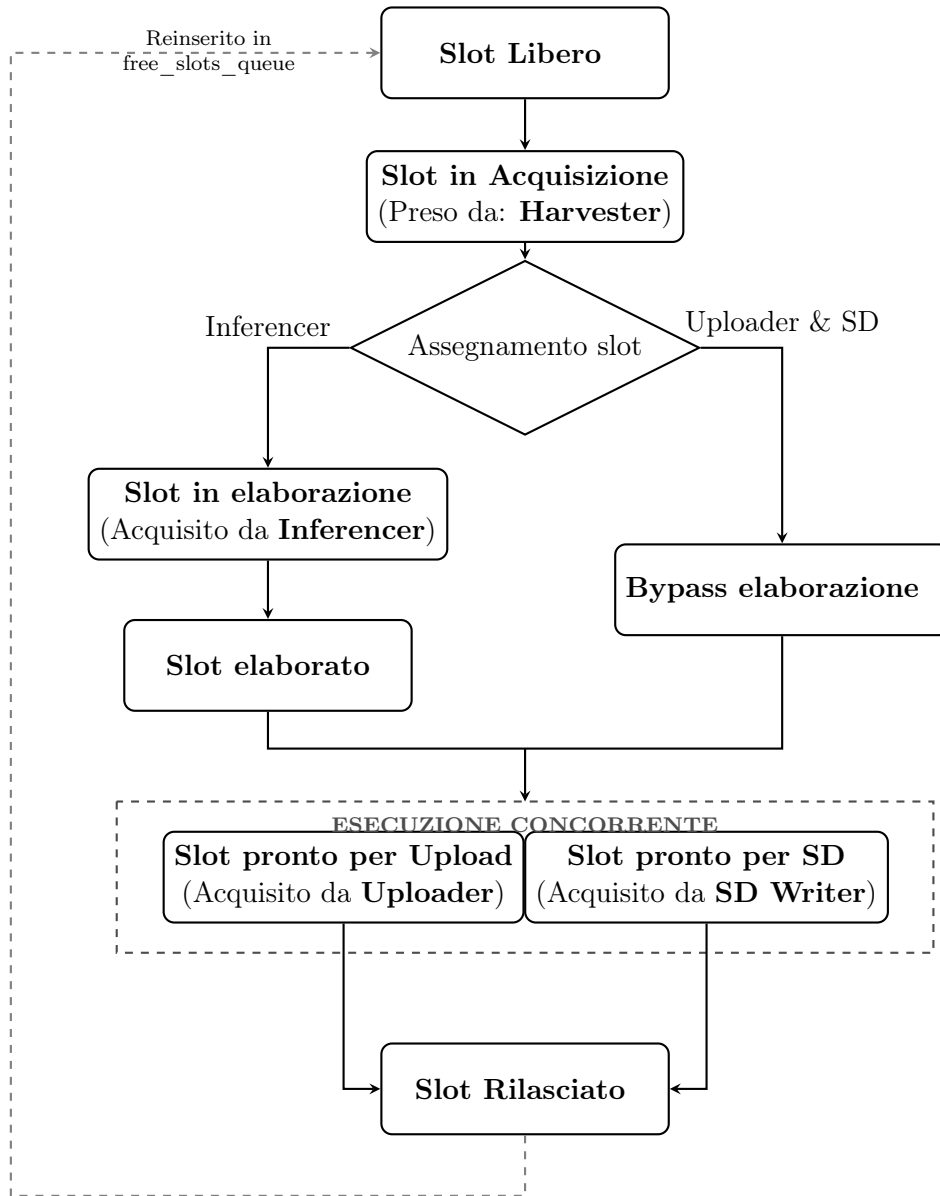


Figura 3.2: Grafo del ciclo di vita di uno slot in modalità continua

L'harvester prende il primo slot che trova quindi, dopo essere stato popolato dai dati, viene acquisito dall'inferencer, e poi sd_writer e uploader salvano su sd e caricano sul cloud il segmento con i dati inferenziali, oppure se l'inferencer sta elaborando un altro slot, viene assegnato direttamente al uploader e al sd_writer, che salvano e caricano lo slot senza dati inferenziali. La ragione per cui abbiamo bisogno di 4 buffer è che questo garantisce che 2 siano sempre occupati dal harvester che salva i dati come un classico ping pong buffer senza interruzioni, uno sia occupato

dall'inferencer e un altro dai uploader e sd_writer, in questo modo nessun task deve aspettarne un altro.

Il sistema é predisposto anche per altre modalità. Le modalità di upload di dati, configurabili dalla backend runtime sono:

- *Continuous*: ovvero quello nel grafico sopra, il device carica sul cloud tutti i dati che riesce, elaborati o no.
- *AnomalyOnly*: il device carica sul cloud il segmento solo se risulta in un anomalia.
- *AnomalyOrPeriodic*: il device carica sul cloud il segmento solo se risulta in un anomalia, ma carica con una certa cadenza anche dati non anomali per permettere un allenamento continuo dei modelli.
- *ContinuousScoreRawAnomaly*: il device carica sempre il risultato dell'inferenza, ma carica anche il segmento solo se si tratta di un anomalia.

Il device espone alla backend dei comandi di dump opzionali, questi sono *dump*, che carica sul cloud tutti i dati dalla scheda SD senza farne l'inferenza (harvester e inferencer disabilitati) e *dump_i* che carica tutti i dati, ma calcolandone anche le anomalie (harvester disabilitato).

Nella configurazione del device é possibile anche disabilitare del tutto la scheda sd per risparmiare energia se non si ha bisogno di chiamare il comando *dump* dei dati.

3.3 Inferencer

L'inferencer é la task che gestisce la vita dei modelli e la loro inferenza, appoggiandosi ad una struttura dati chiamata *ensemble* che aiuta la gestione dei modelli e astrae l'utilizzo della libreria tflite micro [Espressif Systems, 2024d]. Ai dati viene applicata una pipeline che riduce la dimensionalità, ovvero una riduzione euclidea di magnitudo per accelerazione e giroscopio:

$$\text{Accel Magnitude}[n] = \sqrt{\text{accel}_x[n]^2 + \text{accel}_y[n]^2 + \text{accel}_z[n]^2}$$

$$\text{Gyro Magnitude}[n] = \sqrt{\text{gyro}_x[n]^2 + \text{gyro}_y[n]^2 + \text{gyro}_z[n]^2}$$

questo permette di mantenere solo i dati vibrazionali, indipendentemente dall'orientamento di montaggio del sensore.

Successivamente viene applicata ad entrambi una funzione a finestra di hanning (overlap 75%) e una trasformazione di Fourier. Viene scelta la finestra di Hanning perchè offre un buon compromesso tra risoluzione e contenimento dello spectral leakage [Harris, 1978].

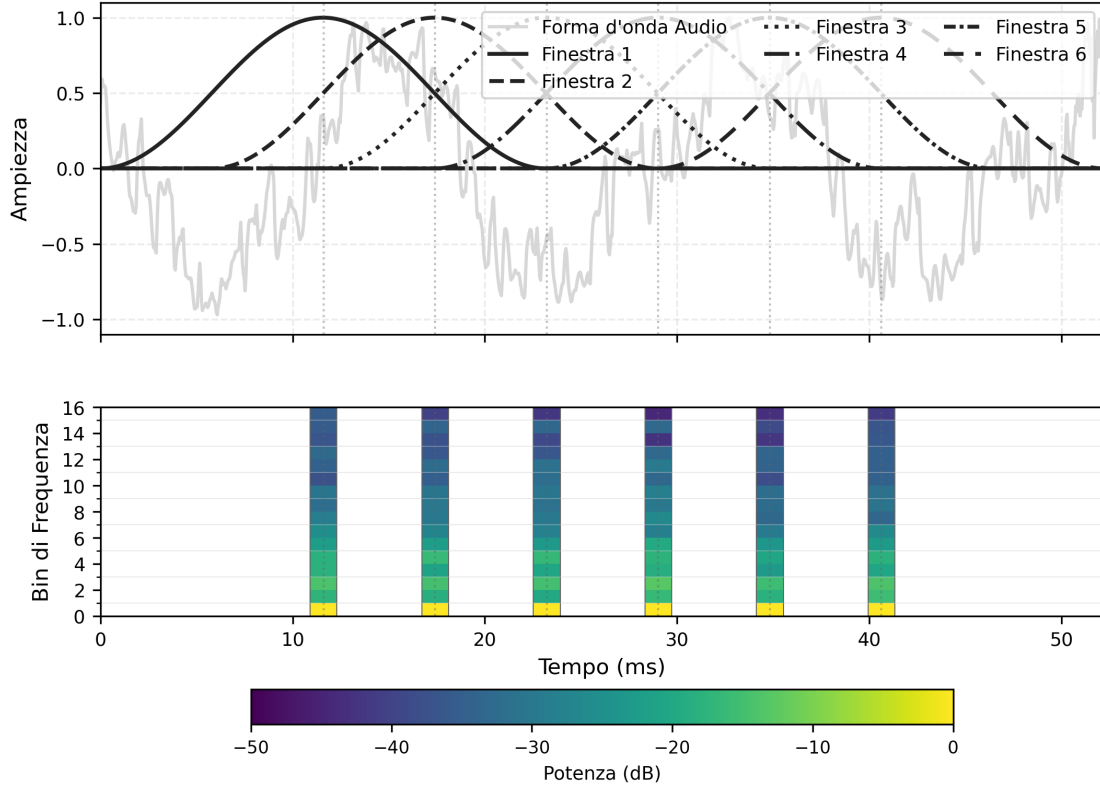


Figura 3.3: Applicazione finestra di hanning e trasformata di fourier

Infine gli spettrogrammi STFT del giroscopio e accelerazione vengono concatenati lungo l'asse delle feature di frequenza, così da creare un solo segmento di dati.

3.3.1 Architettura dell'*ensemble*

L'architettura è formata da 3 tipi di modelli disposti in maniera simil-MIXAE [Zhang et al., 2017]:

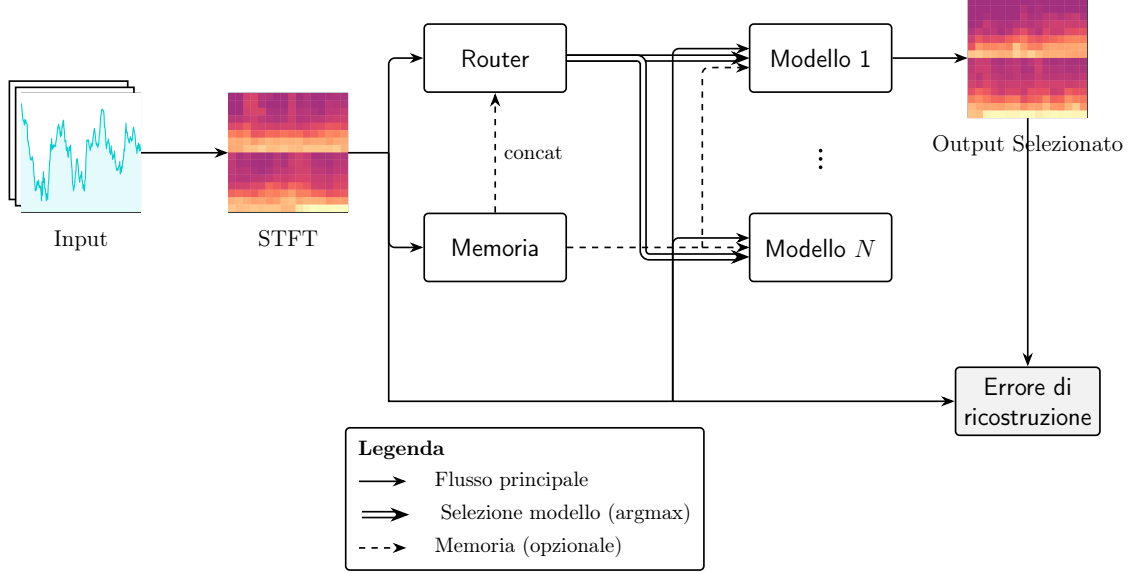


Figura 3.4: Architettura del *ensemble*.

- 1 **Router**: prende in input un segmento e lo stato della memoria e decide quale autoencoder dovrà processarlo.
- 1 **Memoria**: memoria, opzionale per modello. Utile per modelli non convoluzionali, è un modo per aggiungere informazioni sul passato.
- N **autoencoder**: ci sono N (max 16), autoencoder che prendono il segmento e provano a ricostruirlo, l'errore di ricostruzione indica il livello di anomalia, configurabile tra MSE lineare e MSE logaritmico, può essere usato qualsiasi modelli rappresentabile con grafi nativi tflite micro.

3.3.2 Gestione dei sottomodelli

In memoria non possono alloggiare tutti i sottomodelli, se si scelgono architetture troppo complesse, quindi è necessario avere un modo di caricare a bisogno i sottomodelli. Per questo l'inferencer dispone di una struttura dati di cache di modelli, creando così una gerarchia di recupero dati. Prima viene interrogata la

cache alla ricerca del modello, poi la scheda sd se abilitata, poi il cloud attraverso una richiesta MQTT. Una volta caricato un nuovo modello, se la memoria non è sufficiente, viene rimosso il modello usato meno recentemente, liberando così una parte di memoria.

3.4 Ottimizzazioni

Vengono usate varie funzioni ottimizzate per l'ESP32S3.

- **dsp_opt_int16_to_float**, custom, permette di convertire un buffer da int16 a float con l'istruzione `FLOAT.S` di Xtensa LX7 [Cadence Design Systems, 2019].
- **dsps_fft2r_fc32 (Radix-2 Complex FFT)**: calcola la fft sui dati del sensore. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_fft2r_init_fc32 (Inizializzazione FFT)**: Precalcola le tabelle dei fattori rotazionali (*twiddle factors*) per evitare runtime calcoli di seni e coseni, molto pesanti. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_bit_rev_fc32 (Bit Reversal)**: Esegue il riordinamento dell'array in output di `dsps_fft2r_fc32`. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_wind_hann_fc32 (Generazione Finestra di Hanning)**: Calcola e applica la finestra di Hann. Fornita da ESP-DSP [Espressif Systems, 2024a].
- **dsps_mulc_fc32 (Moltiplicazione Vettore-Scalare)**: Esegue la moltiplicazione di un array float per un numero. Fornita da ESP-DSP [Espressif Systems, 2024a].

3.4.1 Ottimizzazione della conversione in float

Il sensore restituisce dati in tipo int16, ma le funzioni di calcolo si aspettano dati in formato float da -1 a 1, quindi dobbiamo convertirli usando la funzione `dsp_opt_int16_to_float` che sfrutta l'istruzione assembly `FLOAT.S`.

L'istruzione `FLOAT.S`

L'approccio *naive* sarebbe fare un ciclo for facendo un cast di ogni valore del buffer in float e poi moltiplicarlo per $1/32768.0$:

$$x_{\text{float}} = (\text{float})x_{\text{int16}} \cdot \frac{1}{32768.0} \quad (3.9)$$

Questo approccio fa eseguire nell'FPU del controllore due istruzioni: un cast hardware (`FLOAT.S` con fattore di scala nullo) ed una moltiplicazione in floating point (`MUL.S`).

Per ottimizzare si è sfruttato l'ISA del coprocessore matematico dell'architettura Xtensa LX7 dell'ESP32-S3. Come scritto nel manuale dell'ISA [Cadence Design Systems, 2019], `FLOAT.S` possiede un campo per fare lo shift del risultato:

$$\text{FR}[r] \leftarrow \text{floats}(\text{AR}[s]) \cdot 2^{-t} \quad \text{con } t \in [0, 15] \quad (3.10)$$

Impostando $t = 15$, la FPU procede col casting int16-float e la divisione per $2^{15} = 32768$ nello stesso ciclo di clock. Si elimina così l'istruzione di moltiplicazione e l'overhead di memoria per la costante.

Ottimizzazione della pipeline

Come definito nei modelli di schedulazione di GCC per l'architettura Xtensa [Free Software Foundation, 2024], le istruzioni di caricamento sulla memoria `load` e di conversione (`fconv`) hanno latenze:

- **Caricamento** (`xtensa_memory`): 2 cicli di clock.
- **Conversione** (`xtensa_fconv`): 2 cicli di clock (fully-pipelined, con throughput di 1 istruzione per ciclo).

Quindi se viene eseguita la conversione con `FLOAT.S` elemento per elemento, ovvero con fattore di unrolling $U=1$, la cpu dovrà continuare ad aspettare che la FPU scriva i risultati nei registri prima di poter salvare il valore in memoria. Per evitare la latenza di pipeline abbiamo bisogno di un fattore di unrolling U maggiore uguale della latenza di pipeline [Hennessy and Patterson, 2017].

La latenza che abbiamo è di 4 cicli (2 di caricamento+2 di conversione), Quindi scegliamo $U=4$.

Lo pseudocodice dell'algoritmo ottimizzato con inline assembly è riportato nel Codice 3.1.

Codice 3.1: Algoritmo di conversione ottimizzato a 4 vie con assembly Xtensa

```
void dsp_opt_int16_to_float(const int16_t *src, float *dest,
    int len) {
    int i = 0;
    for (; i + 3 < len; i += 4) {
        // genera istruzioni L16SI,
        // carica e converte in 2 cicli
        int32_t s0 = src[i+0]; int32_t s1 = src[i+1];
        int32_t s2 = src[i+2]; int32_t s3 = src[i+3];
        float f0, f1, f2, f3;

        asm volatile (
            "float.s_\ %[f0],_\ %[s0],_\ 15_\n\t"
            "float.s_\ %[f1],_\ %[s1],_\ 15_\n\t"
            "float.s_\ %[f2],_\ %[s2],_\ 15_\n\t"
            "float.s_\ %[f3],_\ %[s3],_\ 15_"
            : [f0] "=f" (f0), [f1] "=f" (f1), [f2] "=f" (f2),
              [f3] "=f" (f3)
            : [s0] "r" (s0), [s1] "r" (s1), [s2] "r" (s2),
              [s3] "r" (s3)
        );

        dest[i+0] = f0; dest[i+1] = f1;
        dest[i+2] = f2; dest[i+3] = f3;
    }
}
```

Validazione sperimentale benchmark

Per validare l'analisi è stato condotto un benchmark sull'ESP32-S3 misurando i cicli di clock effettivi tramite il registro CCOUNT. Ogni benchmark è stato fatto su 20000 volte un buffer da 4096 valori, per simulare il vero utilizzo.

Configurazione	Cicli/op	Tempo buffer	Ottimizzazione	Speedup
Assembly x1	17.08	291.50 μ s	29.24%	1.41×
Assembly x2	10.57	180.39 μ s	56.21%	2.28×
Assembly x4	8.26	140.97 μs	65.78%	2.92×
Assembly x8	9.01	153.77 μ s	62.68%	2.68×
Naive C x1	24.14	411.99 μ s	0.00%	1.00×
Naive C x2	17.58	300.03 μ s	27.17%	1.37×
Naive C x4	14.32	244.39 μ s	40.68%	1.68×
Naive C x8	12.82	218.79 μ s	46.90%	1.88×
Naive C x16	11.95	203.95 μ s	50.50%	2.02×
Naive C x32	22.55	384.85 μ s	6.59%	1.07×
Naive C x64	23.34	398.29 μ s	3.33%	1.03×

Tabella 3.8: Risultati del benchmark su ESP32-S3 a 240 MHz

Si può notare che il metodo più veloce è Assembly con $U=4$, il che conferma l'analisi teorica. Si può anche notare che il metodo naive più veloce è con $U=16$, questo perchè la pipeline di calcolo naive è lunga 9 cicli perchè viene scomposta in 4 istruzioni, ovvero L16SI (2 cicli), FLOAT.S (2 cicli, senza shift), MUL.S (4 cicli), SSI (Store Single Precision, 1 ciclo), per un totale di 9 cicli totali. U deve essere quindi maggiore o uguale a 9, tuttavia per allineamento alla dimensione del bus, usiamo la potenza di due maggiore più vicina, ovvero 16.

Capitolo 4

Modelli utilizzati

Come abbiamo detto ci sono 3 tipi di modelli, il modello memoria, il modello router, e N sottomodelli autoencoder.

4.0.1 Modello Memoria

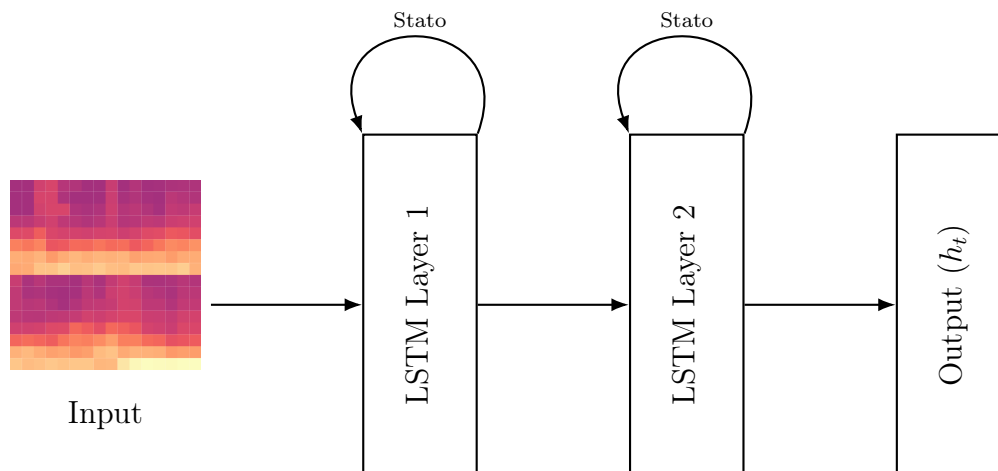


Figura 4.1: Modello a LSTM

LSTM Memory Backbone utilizza dei layer a Long Short Term Memory (LSTM) [Hochreiter and Schmidhuber, 1997], che permettono agli altri sottomodelli ricordare dati passati. Riceve in input i singoli segmenti dello spettrogramma e fornisce in output una rappresentazione latente dei dati. Il modello memoria è condiviso con tutti i modelli che lo usano, quindi gli altri modelli devono essere a dipendenza del modello memoria.

Modello router

Il Router funge da classificatore: riceve lo spettrogramma in input e produce una probabilità sulle N classi associate agli autoencoder. Viene quindi selezionato il sottomodello con la confidenza più alta. Se nessuna uscita supera una soglia minima di confidenza, il segmento viene classificato direttamente come anomalia.

Le architetture usate sono:

1. **Fully connected:** Rete neurale densa (Multilayer Perceptron, MLP) [Rumelhart et al., 1986].
2. **STFT MCNN:** Multi-branch Convolutional Neural Network. Applica 3 reti convoluzionali parallele per estrarre caratteristiche dei dati diverse, usando kernel di dimensioni diverse pari a 3, 5 e 7, da pooling e fusione densa [Zhang et al., 2016].
3. **1D CNN** Rete convoluzionale (multilivello) ad un kernel per livello che scansiona lo spettrogramma, ha un MLP finale per la classificazione. [Kiranyaz et al., 2015].

Modelli autoencoder

Provano a ricostruire il segnale di input, la ragione per cui sono utili nel contesto dell'anomaly detection è che riescono a ricostruire il segnale di input solo se sono stati allenati con dati simili a quelli che dovranno poi ricostruire, questo fa sì che se incontrano dati mai visti non riusciranno a ricostruire in maniera affidabile l'input, e quindi possiamo assumere che quell'input è diverso in struttura dai dati in cui è stato allenato. La formula della ricostruzione è $\hat{x} = f_{dec}(f_{enc}(x))$ e il punteggio di somiglianza $s = \mathcal{L}(x, \hat{x})$.

1. **Autoencoder denso:** Usa semplici layer densi, essendo di fatto un MLP con dei layer di bottleneck, che formano lo spazio latente, tra l'encoder e il decoder.
2. **Variational autoencoder:** Un VAE è, a livello strutturale, un autoencoder denso, tuttavia nella fase di training, si forza lo spazio latente ad essere formato da una distribuzione normale $\mathcal{N}(0, 1)$ di $\lceil$ dimensioni. Questo permette

allo spazio latente di non avere buchi, ovvero zone che producono in fase di decode dati incoerenti [Kingma and Welling, 2013]. In questo caso viene usato come funzione di somiglianza la divergenza KL (Kullback-Leibler), ovvero la somiglianza dello spazio latente alla distribuzione normale oltre all'errore di ricostruzione.

3. **Convolutional Autoencoder 1D:** Usa strati convoluzionali in fase di encode, e strati deconvoluzionali trasposti in fase di decode [Masci et al., 2011]. Utilizza come input lo spettrogramma interpretato come una serie multivariata, dove i singoli bin di frequenza fungono da canali di input. Il filtro convoluzionale scorre esclusivamente lungo l'asse orizzontale.
4. **Convolutional Autoencoder 2D:**
Usa strati convoluzionali in fase di encode, e strati deconvoluzionali in fase di decode [Masci et al., 2011]. Utilizza come input lo spettrogramma interpretato come una vera e propria immagine spaziale a singolo canale.
5. **Convolutional LSTM Autoencoder:** Combina layer convoluzionali (per le feature spaziali) e celle LSTM (per le feature temporali) per catturare anomalie spazio-temporali [Shi et al., 2015].

Capitolo 5

Sviluppo del Backend e Integrazione

5.1 Pipeline dei Dati

Tutti i messaggi che vengono scambiati tra l'edge e il servizio sono codificati con CBOR (2.3), formalizzato da schemi CDDL [Birkholz et al., 2019] condivisi nel progetto.

5.1.1 Topic MQTT

- Uplink

- `v1/{node_id}/data/sensor`: Chunk di campioni raw dei 6 assi IMU, una finestra completa è formata da 4096 valori per asse, il che sarebbe troppo pesante da mandare in un unico pacchetto, quindi la finestra viene separata in chunk più piccoli.

La finestra ha un id incrementale numerico, per poi ricostruire l'ordine cronologico delle finestre, se il sensore notifica un *OVERRUN* significa che il controllore è stato troppo lento a leggerne i dati e viene saltato un id, in questo modo il backend rileva immediatamente la discontinuità temporale.

- `v1/{node_id}/inference`: Risultati dell'inferenza: LogMSE, flag anomalia, id modelli e tempo di inferenza.
- `v1/{node_id}/info/caps`: Capabilities del firmware per quanto riguarda le operazioni TFLite compilate nel firmware, usate per notificare la

backend di quali modelli può usare. Notifica anche le frequenze disponibili del sensore.

- `v1/{node_id}/info/health`: RAM libera, stato scheda SD, modelli attivi in cache e inferenze per secondo.
- `v1/{node_id}/config/fetch`: Richiesta al backend della configurazione runtime attiva.
- `v1/{node_id}/ensemble/fetch`: Richiesta al backend della configurazione dell'ensemble attivo.
- `v1/{node_id}/models/fetch/{type}/{id}`: Richiesta di download del modello TFLite.

- **Downlink**

- `v1/{node_id}/config`: Configurazione del device.
- `v1/{node_id}/ensemble`: Configurazione dell'ensemble.
- `v1/{node_id}/models/{type}/{id}`: Modelli TFLite Micro divisi per tipo router, memory o autoencoder.
- `v1/{node_id}/cmd/reboot`: Riavvio forzato immediato dell'ESP32.
- `v1/{node_id}/cmd/updt_status`: Forza la pubblicazione istantanea del pacchetto health.
- `v1/{node_id}/cmd/dump`: Avvia la lettura e lo streaming continuo dei buffer raw memorizzati nella scheda SD.
- `v1/{node_id}/cmd/dump_i`: Avvia lo streaming dei record e relativa inferenza dei dati registrati nella scheda SD.

5.1.2 Persistenza dei Dati

Il backend sfrutta PostgreSQL [PostgreSQL Global Development Group, 2024] con l'estensione TimescaleDB [Timescale, Inc., 2024], che permette di gestire serie temporali continue con milioni di campioni.



Figura 5.1: Schema relazionale

5.1.3 NAS Engine

Il microservizio `nas_service` implementa una ricerca architetturale dei modelli automatizzata alimentata da *Optuna* [Akiba et al., 2019] e monitorata su *ClearML* [Allegro AI, 2024]. Al posto di addestrare modelli manualmente caso per caso, Optuna esegue uno studio probabilistico con ottimizzazione Bayesiana e campionatore TPE (Tree-structured Parzen) [Bergstra et al., 2011] sui dati storici

caricati dal sensore per scoprire la migliore architettura dell'intero ensemble, scegliendo i sottomodelli e i loro parametri. Optuna deve bilanciare le performance dell'ensemble e la sua accuratezza. I dati per l'allenamento vengono divisi in 80% per il training e 20% per validazione con dati anomali e non anomali, da Grafana [Grafana Labs, 2024] si possono impostare segmenti manualmente come anomalie. Una volta trovata l'architettura migliore, viene salvata nella base di dati e caricata via MQTT sul device.

Parametri di Optuna

Costanti Hardware per optuna:

- Clock CPU a 240 MHz.
- RAM cache di 4MB.
- Velocità di lettura da SD 1MB/s.
- Soglia minima di velocità di inferenza $>0.5\text{Hz}$.

Spazio di Ricerca degli Iperparametri

Durante ogni iterazione di ricerca Optuna può cambiare:

- **Memoria**

Optuna decide autonomamente se il segnale vibrazionale richiede memoria o se è sufficiente un approccio stateless:

- `use_memory`: [False, True]
- Se True ha Iperparametri:
 - * `hidden_dim`: [16, 32, 64, 128] (dimensione dello stato latente della memoria);
 - * `state_dim`: [16, 32, 64] (dimensione dei vettori di stato h e c mantenuti in RAM);
 - * `num_layers`: [1, 2] (profondità della cella LSTM);
 - * `learning_rate`.

- Dimensionamento dell'Ensemble e Router
 - Numero di sottomodelli (`num_routes`): da 1 a 16:
 - * Se `num_routes = 1`: disattiva il router.
 - * Se `num_routes > 1`: campiona l'architettura del Router tra DenseRouter, Conv1DRouter, STFTMCNNRouter.
 - Iperparametri del Router:
 - DenseRouter:
 - * `hidden_units`: [16, 32, 64, 128] neuroni per layer.
 - * `num_layers`: [1, 2, 3].
 - * `learning_rate`: 1e-4 .. 1e-2.
 - Conv1DRouter (Convoluzione 1D + Testa Densa):
 - * `filters`: [8, 16, 32] filtri convoluzionali (kernel 3).
 - * `hidden_units`: [16, 32, 64] neuroni nello strato denso.
 - * `learning_rate`: 1e-4 .. 1e-2.
 - STFTMCNNRouter (Multi-Branch CNN + Fusione Densa):
 - * `branch_filters`: [8, 16, 32] filtri paralleli per ciascuno dei 3 rami (kernel 3, 5 e 7).
 - * `hidden_units`: [16, 32, 64] neuroni dello strato denso finale.
 - * `learning_rate`: 1e-4 .. 1e-2.
 - Iperparametri degli Autoencoder:
 - DenseAutoencoder:
 - * `latent_dim`: [8, 16, 32, 64] dimensione dello spazio latente.
 - * `num_layers`: [1, 2, 3] strati densi di encode/decode.
 - * `skip`: [1, 2, 4, 8] passo di sottocampionamento dei frame, può decidere se saltare *skip* punti di input per velocizzare l'inferenza.
 - * `limit`: [0.05 .. 0.5] soglia di anomalia.
 - VariationalAutoencoder:

- * `temporal_depth` la dimensione di input/output.
- * `latent_dim`: [8, 16, 32] dimensione dello spazio latente gaussiano.
- * `num_layers`: [1, 2, 4] strati densi di encode/decode.
- * `skip`: [1, 2, 4] passo di sottocampionamento temporale dei frame.
- * `limit`: [0.05 .. 0.5] soglia di anomalia.
- **Conv1DAutoencoder** (Convoluzione 1D lungo l'asse delle frequenze):
 - * `filters`: [16, 32, 64] filtri convoluzionali.
 - * `latent_dim`: [16, 32] dimensione del vettore di bottleneck.
 - * `num_layers`: [1, 2] strati di Conv1D e Conv1D-Transpose.
 - * `skip`: [1, 2, 4] passo di sottocampionamento temporale dei frame.
 - * `limit`: [0.05 .. 0.5] soglia di anomalia.
- **Conv2DAutoencoder** (Convoluzione 2D spazio-temporale):
 - * `filters`: [16, 32, 64] filtri convoluzionali 2D.
 - * `latent_dim`: [16, 32] dimensione del bottleneck.
 - * `num_layers`: [1, 2] strati di Conv2D e Conv2D-Transpose.
 - * `skip`: [1, 2, 4] passo di sottocampionamento temporale dei frame.
 - * `limit`: [0.05 .. 0.5] soglia di anomalia.
- **ConvLSTMAutoencoder** (Convoluzione combinata a celle ricorsive LSTM):
 - * `filters`: [16, 32] filtri convoluzionali.
 - * `hidden_dim`: [16, 32] dimensione dello stato ricorsivo della cella LSTM.
 - * `latent_dim`: [16, 32] dimensione del bottleneck.
 - * `skip`: [1, 2, 4] passo di sottocampionamento temporale dei frame.
 - * `limit`: [0.05 .. 0.5] soglia di anomalia.

Capitolo 6

Testing, Deployment e Risultati

6.1 Benchmark

6.1.1 Velocità

È stato eseguito un benchmark per architettura direttamente sull'hardware di destinazione. Per bilanciamento ciascun modello è stato vincolato ad 1 MB di memoria flash, la tabella 6.1 mostra i risultati. Possiamo notare nella tabella che il router MCNN è particolarmente lento e ingombrante, questo perché è sia strutturalmente il più pesante come architettura, sia fortemente penalizzato dal vincolo di memoria a 1 MB. È possibile anche notare che gli autoencoder convoluzionali 2D, o convoluzionali sono particolarmente pesanti, questo perché mentre la conv 1D sfrutta il dominio della frequenza come canale multifeature e comprime esclusivamente la dinamica temporale mantenendo vettori piccoli, la conv 2D tratta lo spettrogramma come una vera matrice spaziale, pagando un prezzo quadratico sia nelle feature map intermedie (RAM di attivazione $\times 11$) sia nelle matrici di proiezione del collo di bottiglia latente (Flash $\times 10$).

Architettura	Memoria	Range flash	RAM [kB]	Latenza [ms]
Router				
Dense	No	66 – 645 KB	8.7	34.22
Dense	Sì	67 – 654 KB	8.9	34.38
Conv 1D	No	18 – 85 KB	9.5	58.90
Conv 1D	Sì	19 – 89 KB	9.7	58.99
STFT MCNN	No	773 KB – 4.0 MB*	385.7	245.24
STFT MCNN	Sì	775 KB – 4.0 MB*	385.9	253.89
Modulo di Memoria Temporale				
LSTM	–	62 – 226 KB	31.2	248.28
Autoencoder				
Denso	No	261 KB – 3.20 MB	8.9	33.45
Denso	Sì	264 KB – 3.22 MB	9.2	33.86
Variazionale	No	263 KB – 1.20 MB	8.9	33.45
Variazionale	Sì	266 KB – 1.21 MB	9.2	33.86
Conv 1D	No	67 – 409 KB	11.6	124.60
Conv 1D	Sì	68 – 412 KB	11.8	125.07
Conv 2D	No	1.04 – 4.0 MB*	129.7	201.18
Conv 2D	Sì	1.04 – 4.0 MB*	129.8	203.91
ConvLSTM	No	553 KB – 2.11 MB	138.2	346.07
ConvLSTM	Sì	559 KB – 2.12 MB	138.6	349.91

Tabella 6.1: Benchmark delle prestazioni di inferenza on-device su ESP32-S3 a 240 MHz (modelli benchmark normalizzati a circa 1 MB di Flash, la latenza e arena sono calcolate su modelli grandi 1MB con Tensor Arena allocata in PSRAM). La colonna *Range flash* riporta la dimensione reale (file `.tflite`) esplorata dal NAS tra configurazione minima e massima, con limite superiore troncato a 4.0 MB (*).

6.1.2 Accuratezza e Valutazione Benchmark

La tabella 6.2 riassume i risultati di rilevamento ottenuti senza memoria e con memoria (formato: *Senza Memoria / Con Memoria*). Il dataset usato è il dataset fornito da Bosch [Bosch Research, 2022], contiene sia dati anomali che non di macchinari CNC in diversi processi.

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(F1-Score: Senza Memoria / Con Memoria)		
Dense AE	0.495 / 0.487	0.489 / 0.486	0.499 / 0.500
Variazionale AE	0.503 / 0.498	0.498 / 0.502	0.510 / 0.506
Conv 1D AE	0.473 / 0.463	0.453 / 0.462	0.467 / 0.475
Conv 2D AE	0.516 / 0.519	0.520 / 0.512	0.526 / 0.521
ConvLSTM	0.428 / 0.415	0.417 / 0.423	0.418 / 0.431
Ensemble Eterogeneo	0.517 / 0.514	0.505 / 0.509	0.517 / 0.515

Tabella 6.2: F1-Score medio su tutte le valutazioni CNC al variare dell'autoencoder e router (*Senza Memoria* / *Con Memoria*).

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(F1-Score: Senza Memoria / Con Memoria)		
Dense AE	0.475 / 0.466	0.455 / 0.462	0.475 / 0.469
Variazionale AE	0.496 / 0.491	0.481 / 0.500	0.497 / 0.492
Conv 1D AE	0.448 / 0.432	0.404 / 0.428	0.418 / 0.452
Conv 2D AE	0.512 / 0.524	0.512 / 0.512	0.520 / 0.509
ConvLSTM	0.410 / 0.390	0.384 / 0.400	0.382 / 0.423
Ensemble Eterogeneo	0.569 / 0.571	0.551 / 0.564	0.572 / 0.567

Tabella 6.3: F1-Score medio nelle lavorazioni in cui l'Ensemble Eterogeneo supera i modelli singoli (33.3% del totale).

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(F1-Score: Senza Memoria / Con Memoria)		
Dense AE	0.518 / 0.525	0.490 / 0.515	0.517 / 0.527
Variazionale AE	0.530 / 0.528	0.505 / 0.536	0.527 / 0.535
Conv 1D AE	0.489 / 0.480	0.443 / 0.487	0.454 / 0.507
Conv 2D AE	0.529 / 0.538	0.525 / 0.534	0.530 / 0.537
ConvLSTM	0.456 / 0.442	0.412 / 0.455	0.417 / 0.465
Ensemble Eterogeneo	0.523 / 0.531	0.493 / 0.527	0.516 / 0.524

Tabella 6.4: F1-Score medio nelle lavorazioni CNC in cui la memoria temporale migliora le prestazioni complessive (10 lavorazioni su 30, 33.3% del totale).

La tabella 6.2 mostra il punteggio medio delle varie architetture di modelli, possiamo vedere che in media il miglior modello é formato da sottomodelli convoluzionali 2D.

La tabella 6.3 mostra il punteggio medio delle varie architetture di modelli ma selezionando il terzo dei casi in cui un ensemble eterogeneo ha il punteggio più alto. I risultati mostrano che un ensemble eterogeneo ha un punteggio F1 in media il +10.50% più alto di modelli omogenei, questo dimostra che un ensemble eterogeneo é una valida architettura con determinati tipi di segnale vibrazionale.

La tabella 6.4 mostra il punteggio medio delle varie architetture di modelli ma selezionando il 33.3% dei casi in cui un modelli di memoria hanno il punteggio più alto. Si può notare che in media un modello con memoria aggiunta migliora il punteggio F1 in media del +3.60% questo mostra che un ensemble con memoria sia lievemente migliore di uno senza memoria riguardo il punteggio F1.

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(Separation Ratio: Senza Memoria / Con Memoria)		
Dense AE	1.58 / 1.59	1.59 / 1.59	1.57 / 1.59
Variazionale AE	1.55 / 1.57	1.56 / 1.57	1.54 / 1.56
Conv 1D AE	1.55 / 1.56	1.55 / 1.56	1.53 / 1.55
Conv 2D AE	1.58 / 1.59	1.60 / 1.59	1.58 / 1.58
ConvLSTM	1.50 / 1.50	1.49 / 1.50	1.48 / 1.50
Ensemble Eterogeneo	1.55 / 1.57	1.55 / 1.56	1.54 / 1.56

Tabella 6.5: Separation Ratio medio(formato: *Senza Memoria / Con Memoria*). Valori più alti indicano una discriminazione maggiore dell'anomalia.

La tabella 6.5 mostra il *Separation Ratio* ($MSE_{anomalo}/MSE_{normale}$), quantifica la distanza tra l'errore di ricostruzione dei segmenti anomali rispetto ai sani.

6.1.3 Occupazione di Memoria RAM e Flash

La tabella 6.6 mostra la RAM necessaria per l'inferenza, mostrando l'impatto della memoria. La tabella 6.7 mostra l'occupazione totale in memoria flash degli ensemble.

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(RAM di Attivazione [kB]: Senza Memoria / Con Memoria)		
Dense AE	656.5 / 741.0	560.5 / 645.1	2065.0 / 2149.6
Variazionale AE	660.5 / 745.1	564.6 / 649.1	2069.1 / 2153.6
Conv 1D AE	257.2 / 341.8	161.3 / 245.8	1665.8 / 1750.3
Conv 2D AE	1198.2 / 1282.8	1102.3 / 1186.8	2606.8 / 2691.3
ConvLSTM	658.0 / 742.5	562.0 / 646.6	2066.5 / 2151.1
Ensemble Eterogeneo	1180.3 / 1264.9	1084.4 / 1168.9	2588.9 / 2673.4

Tabella 6.6: RAM di attivazione [kB] (formato: *Senza Memoria / Con Memoria*)..

Autoencoder	Dense Router	Conv 1D Router	STFT MCNN Router
	(Storage Flash Totale [kB]: Senza Memoria / Con Memoria)		
Dense AE	3284.3 / 3368.8	3188.3 / 3272.9	4692.8 / 4777.4
Variazionale AE	3308.6 / 3393.2	3212.7 / 3297.3	4717.2 / 4801.8
Conv 1D AE	888.8 / 973.3	792.8 / 877.4	2297.3 / 2381.9
Conv 2D AE	6534.8 / 6619.4	6438.9 / 6523.4	7943.4 / 8027.9
ConvLSTM	3293.3 / 3377.8	3197.3 / 3281.9	4701.8 / 4786.4
Ensemble Eterogeneo	4538.9 / 4623.5	4443.0 / 4527.5	5947.5 / 6032.0

Tabella 6.7: Occupazione totale dello storage Flash (in kB) per ciascun modello. Formato: *Senza Memoria / Con Memoria*.

Capitolo 7

Conclusioni e Sviluppi Futuri

7.1 Conclusioni

- **Dimostrazione di fattibilità TinyML su ESP32-S3:** é stata dimostrata la capacità inferenziale di un microcontrollore a basso costo con performance apprezzabili on edge.
- **Dimostrazione delle capacità dell'architettura:** sebbene migliorabile, é stata dimostrata la capacità dell'architettura dell'ensemble eterogeneo per anomaly detection multi-regime.
- **Efficacia della pipeline dai dati al modello:** l'architettura a microservizi si é dimostrata efficace per raccogliere i dati dai sensori e distribuire i modelli allenati ai device.

Come visto nei risultati un *ensemble* eterogeneo può prendere i fattori migliori di ogni autoencoder e router, tuttavia un ensemble eterogeneo non sempre é il modello migliore e la memoria non sempre migliora la qualità in modo significativo.

7.1.1 Quando la Memoria Peggiora

L'analisi mostra che l'uso di un modulo di memoria temporale non introduce un miglioramento sistematico delle performance dei modelli. Motivi fisici per cui la memoria peggiora:

- **Error Smearing:** il modello ricorrente (memoria) accumula informazioni nel suo stato interno. Durante un cambio repentino dei regime, come quando un utensile viene inserito nel materiale, il modello ricorrente ci mette svariati momenti per cambiare lo stato interno con il nuovo regime, l'inerzia rende il modello meno reattivo peggiorando le performance in macchinari soggetti a cambi netti aumentando i Falsi Negativi. Al contrario quando si passa da uno stato rumoroso ad uno stato normale il modello continuerà a fornire Falsi Positivi per qualche frame. Questo migliora la precisione in macchinari con un lento cambiamento di regime, evitando anche che colpi al macchinario dati dagli operatori vengano interpretati come anomalie.
- **Fasi di lavoro discontinue:** un macchinario CNC, come quelli del dataset, non lavora in modo continuo, fa passaggi netti tra uno stato e l'altro, come quando c'è un cambio di direzione. Una memoria LSTM prova a prevedere il prossimo segmento, quando il prossimo segmento cambia completamente regime di lavoro questo viene interpretato come un'anomalia, in questi casi basta una fotografia istantanea del segnale, senza memoria.
- **Vincoli TinyML:** per mantenere una latenza e una dimensione apprezzabili sul device la dimensione del modello deve essere relativamente piccola. Un modello LSTM più piccolo più fatica a trovare dinamiche complesse del segnale.

Tuttavia non tutte le operazioni nel dataset sono di questo tipo, possiamo vedere nella tabella 6.4 che in tipi specifici di operazioni su macchinari, aggiungere la memoria migliora il modello.

7.1.2 Quando un Ensemble Eterogeneo Peggiora

L'analisi mostra che l'uso di un ensemble eterogeneo non introduce un miglioramento sistematico delle performance rispetto a un ensemble omogeneo. Il router è identico e gestisce i segmenti nello stesso modo. I motivi delle performance peggiori sono:

- **Bias di selezione non supervisionata:** In fase di addestramento non supervisionato, viene scelto l'autoencoder migliore, non avendo dati anomali viene scelto il modello più capace a ricostruire il segnale non anomalo, tuttavia

questo modello non é detto che sia adatto alla ricostruzione di segnali anomali, potrebbe essere troppo capace, aumentando di Falsi Negativi.

- **Disomogeneità delle architetture:** il continuo cambiamento di sottomodelli con scale di ricostruzione diverse rende la funzione loss di ricostruzione meno uniforme rispetto ad un ensemble omogeneo, comprimendo il separation ratio complessivo.

Possiamo vedere comunque nella tabella 6.3 che un ensemble eterogeneo é comunque migliore in molti casi, rendendolo una scelta ottima in circa un terzo dei dati nel dataset.

7.2 Sviluppi Futuri

7.2.1 Miglioramento dei modelli

Generazione di dati anomali

Per mitigare il bias di selezione non supervisionata, una tecnica promettente è quella di aggiungere ai dati non anomali disponibili delle perturbazioni artificiali per selezionare i modelli più adatti all'ensemble eterogeneo. Attualmente il calcolo di perturbazioni realistiche non è triviale. Se si prova ad aggiungere rumore bianco, viene distribuita dell'energia nel segnale in modo uniforme su tutti i bin di frequenze. Una rete convoluzionale riesce a separare il rumore artificiale relativamente facilmente, tuttavia sacrificando precisione nel resto delle ricostruzioni senza aver appreso come si verificano i guasti. Se si prova a mascherare una parte di un segmento, azzerandolo o copiandolo da un altro segmento, la rete impara a riconoscere la discontinuità, non l'anomalia fisica.

Per creare segnali anomali artificiali, bisognerebbe modellare matematicamente fenomeni specifici al macchinario, come la scheggiatura di un dente, che aggiungerebbe bande laterali attorno alla frequenza di taglio. Per sintetizzare l'anomalia bisognerebbe conoscere a priori il numero di denti, velocità istantanea, carico di taglio e funzione risposta impulsiva del macchinario. Questo implica che la gestione della generazione dei dati è estremamente specifica al tipo di macchinario.

Gestione della deriva delle vibrazioni

Durante la vita di un macchinario vengono viste derive meccaniche causate da usure progressive naturali cambiando molto lentamente la firma vibrazionale del macchinario. Si propone quindi l'introduzione di una soglia configurabile di deriva, oltre la quale il macchinario viene considerato troppo usurato e necessita di una manutenzione preventiva. Una strategia efficace prevede l'affiancamento di due ensemble complementari, uno fisso nel tempo e uno con continuo allenamento. Quello fisso verrebbe allenato una volta durante l'installazione o dopo una manutenzione, mentre l'altro verrebbe continuamente allenato sui nuovi dati. Il modello fisso nota le derive meccaniche lente, mentre quello continuo le rotture del macchinario, che hanno un cambio repentino dei dati nel breve periodo.

Aggiunta e Unione di Altri Dati

L'architettura può essere estesa prevedendo l'integrazione in modo modulare di altri dati disponibili specifici al macchinario, come temperature di varie parti del macchinario, assorbimento di corrente del motore mandrino o potenza assorbita via Modbus/OPC-UA.

Al fine di incrementare l'accuratezza generale, un'estensione promettente prevede l'introduzione di un monitoraggio distribuito e cooperativo, condividendo tra più sensori i loro dati di uno stesso macchinario, ma in locazioni diverse e con tipi di dato diversi, per insegnare ai modelli il cambiamento vibrazionale tra le varie sezioni del macchinario. La backend attualmente supporta più sensori, ma non in modo collaborativo.

7.2.2 Miglioramento Architettura del Sistema

Architettura Scalabile

Attualmente la architettura della backend é modulare, sfruttando 12 container docker, il che rende la architettura semi scalabile orizzontalmente. Per migliorare la scalabilità orizzontale bisognerebbe passare ad un orchestratore di container, come Kubernetes, per avere più container dello stesso tipo nello stesso momento distribuendo l'allenamento dei modelli.

Tiering dei Dati

Tutta la telemetria grezza viene salvata direttamente nelle hypertable di TimescaleDB. Salvare dati vibrazionali continui genera decine di gigabyte al giorno, rallentando le basi di dati. Per rendere più veloce il database si può inserire un sistema di tiering dei dati, ovvero avere un secondo tier di dati per salvare dati storici salvandoli in un Object storage, come S3 o MinIO in modo compresso.

Sicurezza

Il backend compila il binario .tflite e lo spedisce via MQTT. Un attaccante potrebbe intercettare il modello o il topic MQTT e iniettare un modello malevolo con lo scopo di nascondere guasti e portare all'ente danni economici. Per uno sviluppo futuro si

propone di sfruttare lo standard COSE [Schaad, 2022], supportato direttamente da CBOR, per firmare digitalmente i modelli e i dati per verificarne la sorgente per avere un sistema di Zero-Trust IoT.

Active Learning

Attualmente la backend fornisce un modo grafico visione dei dati e dei modelli attraverso Grafana, tuttavia é poco amichevole per un operatore di un macchinario. Bisognerebbe aggiungere un interfaccia per gli HMI dei macchinari agganciandosi al OPC-UA/Modbus integrato nel PLC del macchinario per esporre un interfaccia semplice per l'operatore per notificare falsi positivi o negativi e sfruttare questi dati nel training essendo manualmente etichettati.

Bibliografia

- [Adafruit Industries, 2020] Adafruit Industries (2020). *Adafruit 4682 Micro SD SPI or SDIO Card Breakout Board*. Adafruit Industries. Scheda breakout passiva a 3.3V priva di LDO e level shifter.
- [Adriansyah et al., 2026] Adriansyah, S., Sumarno, S., Nurfuha, U., Giffari, A., Khoirunisa, T., and Subarkah, W. (2026). Performance analysis of mqtt and http protocols on low-power esp32 devices for iot applications. *CoreID Journal*, 4:38–45.
- [Advanced Monolithic Systems, 2012] Advanced Monolithic Systems (2012). *AMS1117: 1A Low Dropout Voltage Regulator Datasheet*. Advanced Monolithic Systems. Riferimento per la corrente di quiescenza (I_q) tipica dei regolatori LDO generici.
- [Akiba et al., 2019] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- [Allegro AI, 2024] Allegro AI (2024). ClearML: Open-source mlops suite for experiment management, orchestration, and data pipelines. <https://clear.ml/>.
- [Apache Software Foundation, 2011] Apache Software Foundation (2011). Apache Avro specification. <https://avro.apache.org/docs/current/specification/>.
- [Barry and Amazon Web Services, 2022] Barry, R. and Amazon Web Services (2022). *FreeRTOS Reference Manual*. Amazon Web Services.
- [Bergstra et al., 2011] Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554.

- [Birkholz et al., 2019] Birkholz, H., Bormann, C., and Vigano, C. (2019). Concise Data Definition Language (CDDL): A notational convention to express CBOR and JSON data structures. RFC 8610, RFC Editor.
- [Bormann and Hoffman, 2020] Bormann, C. and Hoffman, P. E. (2020). Concise Binary Object Representation (CBOR). RFC 8949, RFC Editor.
- [Bosch Research, 2022] Bosch Research (2022). CNC Machining Data: Real-world industrial vibration dataset for brownfield CNC milling machines. https://github.com/boschresearch/CNC_Machining. Companion repository to [Tnani et al., 2022].
- [Bray, 2017] Bray, T. (2017). The JavaScript Object Notation (JSON) data interchange format. RFC 8259, RFC Editor.
- [Bray et al., 2008] Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., and Yergeau, F. (2008). Extensible Markup Language (XML) 1.0 (fifth edition). W3C recommendation, W3C.
- [Breque et al., 2021] Breque, M., De Nul, L., and Petridis, A. (2021). Industry 5.0: Towards a sustainable, human-centric and resilient european industry. Technical report, European Commission, Directorate-General for Research and Innovation.
- [Cadence Design Systems, 2019] Cadence Design Systems (2019). *Xtensa Instruction Set Architecture (ISA) Reference Manual*. Cadence Design Systems, Inc., rev. 2.0 edition. Chapter 8 (Floating-Point Coprocessor Option), p. 431.
- [David et al., 2021] David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., Warden, P., and Rhodes, R. (2021). TensorFlow Lite Micro: Embedded machine learning for TinyML systems. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pages 800–811.
- [Di Mattia et al., 2019] Di Mattia, F., Galeone, P., De Simoni, M., and Ghelfi, E. (2019). A survey on GANs for anomaly detection.
- [EMQ Technologies Inc., 2024] EMQ Technologies Inc. (2024). EMQX: The open-source distributed MQTT broker. <https://www.emqx.io/>. Accessed: 2026-08-25.
- [Espressif Systems, 2024a] Espressif Systems (2024a). Esp-dsp: Espressif dsp library. <https://github.com/espressif/esp-dsp>. Repository ufficiale GitHub.

- [Espressif Systems, 2024b] Espressif Systems (2024b). *ESP-IDF Programming Guide*. Espressif Systems.
- [Espressif Systems, 2024c] Espressif Systems (2024c). Esp-mqtt api reference - esp32. ESP-IDF Programming Guide.
- [Espressif Systems, 2024d] Espressif Systems (2024d). esp-tflite-micro. <https://components.espressif.com/components/espressif/esp-tflite-micro>.
- [Espressif Systems, 2024e] Espressif Systems (2024e). *ESP32-S3 Series Datasheet*. Espressif Systems. Documentazione ufficiale per consumi e caratteristiche radio/sleep.
- [Espressif Systems, 2024f] Espressif Systems (2024f). espressif/cbor: Cbor library for esp-idf. <https://components.espressif.com/components/espressif/cbor>. Componente per ESP-IDF basato su TinyCBOR.
- [Free Software Foundation, 2024] Free Software Foundation (2024). Gnu compiler collection (GCC) target description for Xtensa (xtensa.md). <https://github.com/gcc-mirror/gcc/blob/master/gcc/config/xtensa/xtensa.md>. Accessed: August 2026.
- [Furuhashi, 2013] Furuhashi, S. (2013). MessagePack: It's like JSON, but fast and small. <https://msgpack.org/>.
- [Grafana Labs, 2024] Grafana Labs (2024). Grafana: The open and composable observability and data visualization platform. <https://grafana.com/>.
- [Harris, 1978] Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete fourier transform. *Proceedings of the IEEE*, 66(1):51–83.
- [Hennessy and Patterson, 2017] Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6th edition.
- [Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- [Intel Corporation and Macieira, 2015] Intel Corporation and Macieira, T. (2015). Tincbor: Concise binary object representation (cbor) library. <https://github.com/intel/tincbor>.
- [International Organization for Standardization, 1994] International Organization for Standardization (1994). Information Technology – Open Systems

- Interconnection – Basic Reference Model: The Basic Model. Standard ISO/IEC 7498-1:1994, International Organization for Standardization, Geneva, Switzerland.
- [Kingma and Welling, 2013] Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- [Kingston Technology, 2024] Kingston Technology (2024). *Industrial microSD Card*. Kingston Technology. Accessed: 2026-08-25.
- [Kiranyaz et al., 2015] Kiranyaz, S., Ince, T., and Gabbouj, M. (2015). Real-time patient-specific ecg classification by 1-d convolutional neural networks. *IEEE Transactions on Biomedical Engineering*, 63(3):664–675.
- [Lyons, 2010] Lyons, R. G. (2010). *Understanding Digital Signal Processing*. Prentice Hall, 3rd edition.
- [Masci et al., 2011] Masci, J., Meier, U., Cireşan, D., and Schmidhuber, J. (2011). Stacked convolutional auto-encoders for hierarchical feature extraction. In *International Conference on Artificial Neural Networks*, pages 52–59. Springer.
- [McNally, 2025] McNally, W. (2025). *The CBOR, dCBOR, and Gordian Envelope Book*. Blockchain Commons.
- [Melichov, 2024] Melichov, M. (2024). Anomaly-detection. <https://github.com/maxmelichov/Anomaly-Detection>.
- [Merkel, 2014] Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2.
- [MongoDB, Inc., 2009] MongoDB, Inc. (2009). BSON (binary JSON) serialization format. <https://bsonspec.org/>.
- [Ng et al., 2011] Ng, A. et al. (2011). Sparse autoencoder. Technical report, Stanford University.
- [PostgreSQL Global Development Group, 2024] PostgreSQL Global Development Group (2024). *PostgreSQL: The World’s Most Advanced Open Source Relational Database*. PostgreSQL Global Development Group. Accessed: 2026-08-25.
- [Ramírez, 2018] Ramírez, S. (2018). FastAPI: Modern, fast (high-performance), web framework for building apis with python. <https://fastapi.tiangolo.com/>.
- [Rifai et al., 2011] Rifai, S., Vincent, P., Muller, X., Glorot, X., and Bengio, Y. (2011). Contractive auto-encoders: Explicit invariance during feature extraction. In *Proceedings of the 28th International Conference on Machine Learning*, pages 833–840.

- [Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- [Saha et al., 2022] Saha, S. S., Sandha, S. S., and Srivastava, M. (2022). Machine learning for microcontroller-class hardware: A review. *IEEE Sensors Journal*, 22(22):21362–21390.
- [Schaad, 2022] Schaad, J. (2022). CBOR Object Signing and Encryption (COSE): Structures and process. RFC 9052, RFC Editor.
- [SD Association, 2022] SD Association (2022). *SD Specifications Part 1 Physical Layer Simplified Specification*. SD Association. Version 9.00.
- [Shi et al., 2015] Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., and Woo, W.-c. (2015). Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 802–810.
- [STMicroelectronics, 2024a] STMicroelectronics (2024a). *ISM330BX: 6-axis IMU with High-Bandwidth Accelerometer, Sensor Fusion and AI for Industrial Applications*. STMicroelectronics. Datasheet di specifica elettrica e registri operativi.
- [STMicroelectronics, 2024b] STMicroelectronics (2024b). *STEVAL-MKI245KA: Evaluation Board for ISM330BX 6-Axis Inertial Module*. STMicroelectronics.
- [Timescale, Inc., 2024] Timescale, Inc. (2024). TimescaleDB: An open-source time-series sql database optimized for fast ingest and complex queries. <https://www.timescale.com/>.
- [Tnani et al., 2022] Tnani, M.-A., Feil, M., and Diepold, K. (2022). Smart data collection system for brownfield CNC milling machines: A new benchmark dataset for data-driven machine monitoring. *Procedia CIRP*, 107:131–136.
- [Turkish Technology Team Foundation (T3), 2024] Turkish Technology Team Foundation (T3) (2024). Power consumption (esp32-s3).
- [Varda, 2008] Varda, K. (2008). Protocol Buffers: Google’s data interchange format. <https://protobuf.dev/>.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need.

- In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008.
- [Vincent et al., 2008] Vincent, P., Larochelle, H., Bengio, Y., and Manzagol, P.-A. (2008). Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 1096–1103.
- [Xu et al., 2024] Xu, Z., Zeng, A., and Xu, Q. (2024). FITS: Modeling time series with 10k parameters. In *International Conference on Learning Representations (ICLR)*.
- [Zhang et al., 2017] Zhang, D., Sun, Y., Eriksson, B., and Balzano, L. (2017). Deep unsupervised clustering using mixture of autoencoders. *arXiv preprint arXiv:1712.07788*.
- [Zhang et al., 2016] Zhang, Y., Zhou, D., Chen, S., Gao, S., and Ma, Y. (2016). Single-image crowd counting via multi-column convolutional neural network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 589–597.
- [Øyvind Rønningstad and Nordic Semiconductor, 2021] Øyvind Rønningstad and Nordic Semiconductor (2021). zcbor: A low-footprint cbor library and code generator for c. <https://github.com/NordicSemiconductor/zcbor>.